

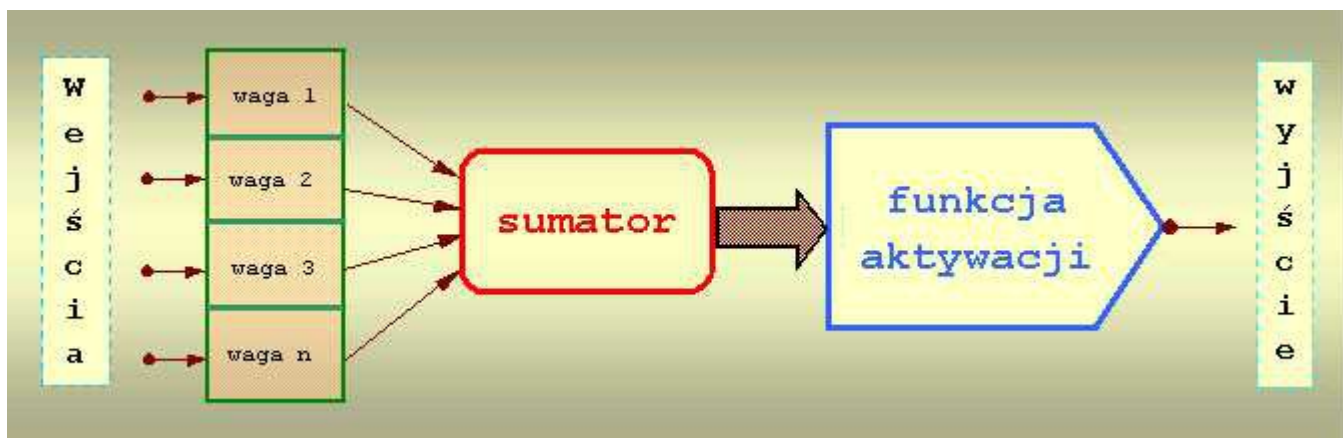
1) Podstawy teoretyczne

Siecią neuronową określa się symulatory (programowe lub sprzętowe) modeli matematycznych realizujące pseudorównoległe przetwarzanie informacji, składające się z wielu wzajemnie połączonych neuronów i naśladujący działanie biologicznych struktur mózgowych.

Podstawowym elementem składowym sieci neuronowej jest:

Neuron

Jego schemat został opracowany przez McCullocha i Pittsa w 1943 roku i oparty został na budowie komórki nerwowej.



Jego działanie jest następujące:

Do wejść doprowadzane są sygnały dochodzące z neuronów warstwy poprzedniej. Każdy sygnał mnożony jest przez odpowiadającą mu wartość liczbową zwaną wagą. Wpływa ona na percepcję danego sygnału wejściowego i jego udział w tworzeniu sygnału wyjściowego przez neuron. Waga może być pobudzająca - dodatnia lub opóźniająca – ujemna. Jeżeli nie ma połączenia między neuronami to waga jest równa zero. Zsumowane iloczyny sygnałów i wag stanowią argument funkcji aktywacji neuronu.

2) Cel ćwiczenia:

Ćwiczenie polegało na porównaniu efektywności uczenia sieci o zadanej wielkości i o zadanym algorytmie.

a) W pierwszej części ćwiczenia zapoznawaliśmy się z sieciami neuronowymi, które mogą oszacować średnią cenę domu w dzielnicy opisanej przez trzynastę cech demograficznych:

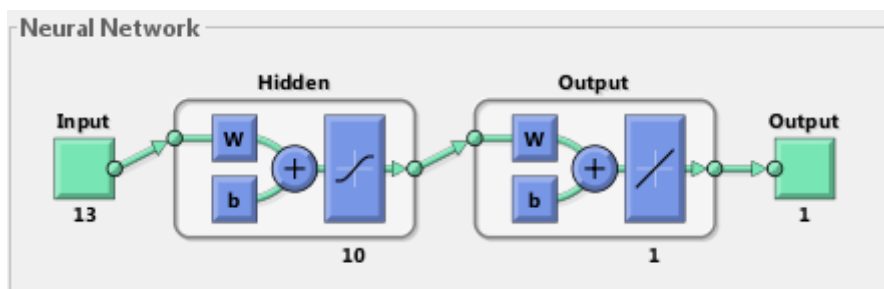
1. Wskaźnik przestępczości na miasto
2. Udział gruntów mieszkalnych ponad 25.000 m kw..
3. Udział niedetalicznych hektarów biznesowych na miasto
4. 1, jeżeli granice dróg Charles River, inaczej 0 Stężenie tlenków azotu
5. (części na 10 mln)
6. Średnia liczba pokoi na mieszkanie
7. Udział jednostek właścicieli mieszkań wybudowanych przed 1940
8. ważonej odległości do pięciu centr zatrudnienia w Bostonie
9. Wskaźnik dostępności do autostrad
10. Pełny kurs wartości nieruchomości opodatkowany za 10.000 dolarów

11. stosunek ucznia do nauczyciela w miastach
12. $1000 (B_k - 0,63)^2$
13. Niższy procent statusu ludności

Przykład ten pozwolił zapoznać się z problemem, gdzie wejścia są dopasowane do związanych wyjść docelowych.

W wyświetlonych oknach przeanalizować można było zadane przez nas parametry, oraz przyciski wyświetlające poszczególne schematy z wynikami doświadczenia.

– Neural Network - schemat sieci neuronowej



– Algorithms:

Algorithms	
Data Division:	Random (dividerand)
Training:	Levenberg-Marquardt (trainlm)
Performance:	Mean Squared Error (mse)
Calculations:	MATLAB

Data Division – czyli podział danych (przypadkowy)

Training – informacja o algorytmie szkoleniowym (Levenberg-Marquardt)

Performance – błąd średnio-kwadratowy w kolejnych epokach

(epoka – jeden etap uczenia)

Calculations – program obliczeniowy (MATLAB)

– Progress

Progress	
Epoch:	0 16 iterations 1000
Time:	0:00:01
Performance:	2.14e+03 8.69 0.00
Gradient:	5.99e+03 9.32 1.00e-07
Mu:	0.00100 0.0100 1.00e+10
Validation Checks:	0 6 6

Epoch – epoka – etap uczenia sieci neuronowej

Time – czas

Performance – błąd średnio-kwadratowy w kolejnych epokach

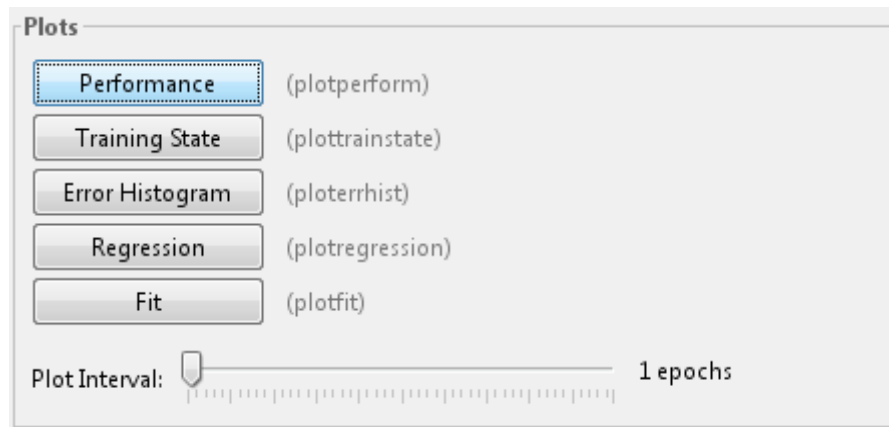
Gradient - różnica, miara wzrostu lub spadku

Mu – współczynnik uczenia

Validation Checks – algorytm sprawdzający czy dla zbioru walidacyjnego błąd też się zmniejsza.

Jeśli się nie zmniejsza w 6 krokach to kończy uczenie.

– Plots



Performance – błąd średnio-kwadratowy w kolejnych epokach przedstawiony w skali logarytmicznej.

Training State – wartość stanu szkolenia

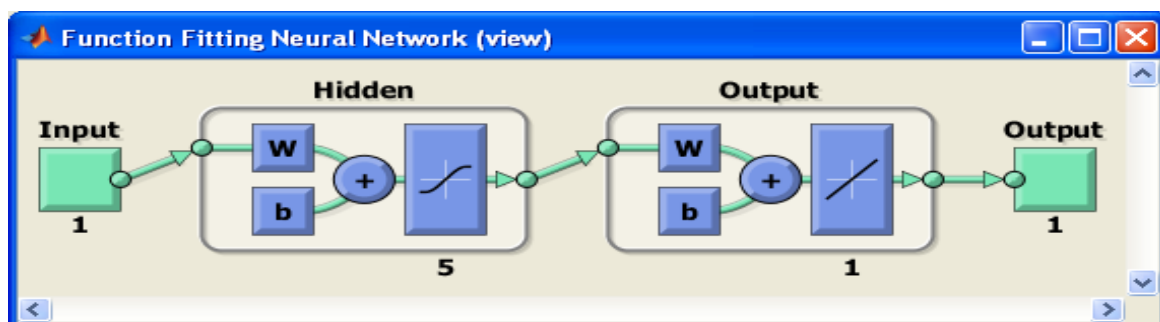
Error Histogram – histogram błędów

Regression – regresja liniowa. Pokazuje czy sieć nauczyła się dobrze zmieniać dane.

Fit – charakterystyka układu

b) Kolejnym etapem ćwiczenia było porównanie efektywności uczenia sieci o zadanej wielkości i o zadanym algorytmie.

Schemat sieci:



Budowa sieci:

Sieć składa się z jednego wejścia i wyjścia, warstwy ukrytej zawierającej 5 neuronów ukrytych o nieliniowej funkcji aktywacji i z 1 neuronem w warstwie wyjściowej, który ma liniową funkcję aktywacji.

Zastosowane funkcje:

Zastosowano cztery funkcje uczenia pozwalające porównać dane:

- algorytm Levenberga-Marquardta
- metoda propagacji wstecznej błędu z momentum
- metoda gradientowa Broydena-Fletcher-Goldfarb-Shanno (quasi-Newtonowska)
- metoda skalowanego gradientu sprzężonego

Algorytm Levenberga-Marquardta

Działanie algorytmu Levenberga-Marquardta opiera się na założeniu że modelowana przez sztuczną sieć neuronową rzeczywista funkcja odwzorująca sygnał wejściowy w sygnał wyjściowy ma charakter liniowy. Takie założenie umożliwia analityczne wyznaczenie minimum funkcji, ale ponieważ rzeczywista funkcja nie jest liniowa więc wyznaczona wartość minimum niekoniecznie jest faktycznym minimum.

Dlatego w kolejnym kroku - po wyznaczeniu hipotetycznego minimum funkcji błędu - algorytm przeprowadza jego testowanie. Jeśli wartość błędu jest mniejsza od błędu w punkcie startowym (poprzednim), to współrzędne nowego punktu wyznaczają nowy punkt startowy, a odpowiadający mu zestaw wag jest traktowany jako nowe przybliżenie optymalnych parametrów uczonej sieci. Proces ten jest powtarzany w kolejnych epokach.

Algorytm Levenberga-Marquardta umiejętnie łączy dwie metody: metodę największego spadku oraz omówiony powyżej algorytm hipotetycznej aproksymacji liniowej (metoda Newtona). W trakcie swojej pracy algorytm wybiera metodę dającą w danej chwili lepsze rezultaty - próbuje hipotetycznej aproksymacji liniowej i jeżeli nie daje ona pozytywnych wyników cofa się do metody największego spadku, co w sumie umożliwia mu bardzo szybkie działanie.

Modyfikacja wag odbywa się wg formuły:

$$\Delta \mathbf{w}(t) = -(\mathbf{H} + \lambda \mathbf{I})^{-1} \left(\frac{\partial \mathcal{E}(\mathbf{w}(t-1))}{\partial \mathbf{w}} \right)$$

gdzie: $\mathbf{H}$ – hesjan funkcji celu

$\mathbf{I}$ – macierz jednostkowa

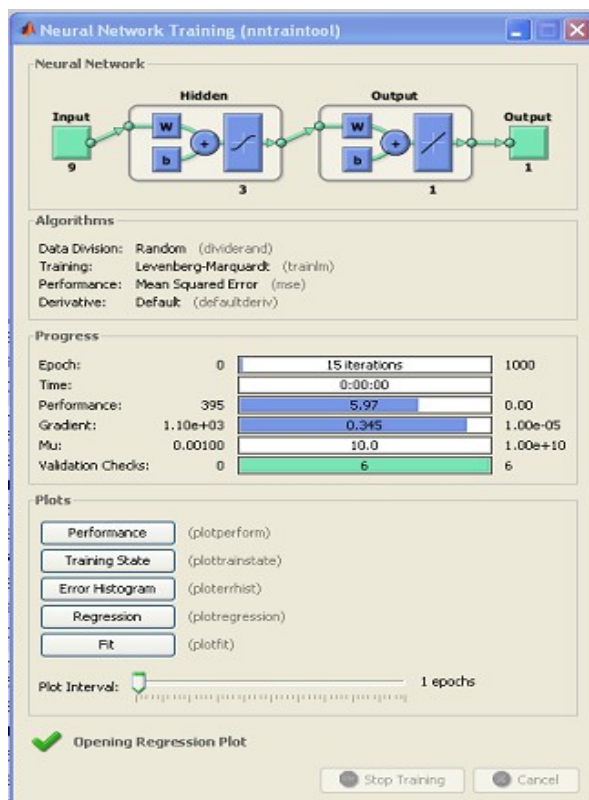
λ – paramet Marquardta

- λ duże (daleko od minimum) – metoda największego spadku
- λ małe (blisko od minimum) – metoda Newtona

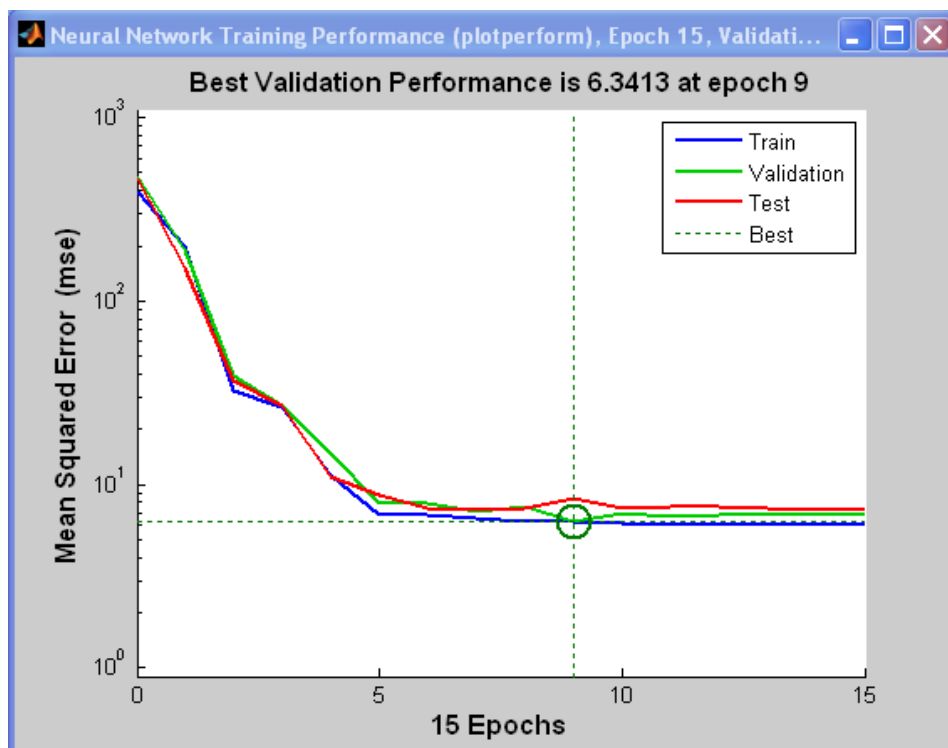
Pierwszy człon w formule Levenberga-Marquardta (zawierający hesjan) odpowiada założeniu o liniowości; drugi metodzie gradientowej. Parametr Marquardta λ określa, która z metod ma charakter dominujący. Za każdym razem, gdy następuje zmniejszenie wartości błędu, wartość parametru jest zmniejszana, wzmacnia to założenie o liniowości i kieruje algorytm w stronę próby bezpośredniego osiągnięcia minimum. Za każdym razem, gdy nie powiedzie się próba zmniejszenia błędu, wartość parametru jest zwiększana. Zwiększa to wpływ metody gradientowej i zmniejsza wielkość kroku. W ten sposób zapewnione jest zmniejszenie wartości błędu niezależnie od położenia aktualnego punktu i wynikającego z niego lepszego lub gorszego przybliżania rzeczywistej funkcji przez jej liniową aproksymację.

Algorytm Levenberga-Marquardta jest jednym z najbardziej efektywnych algorytmów uczenia sieci jednokierunkowych.

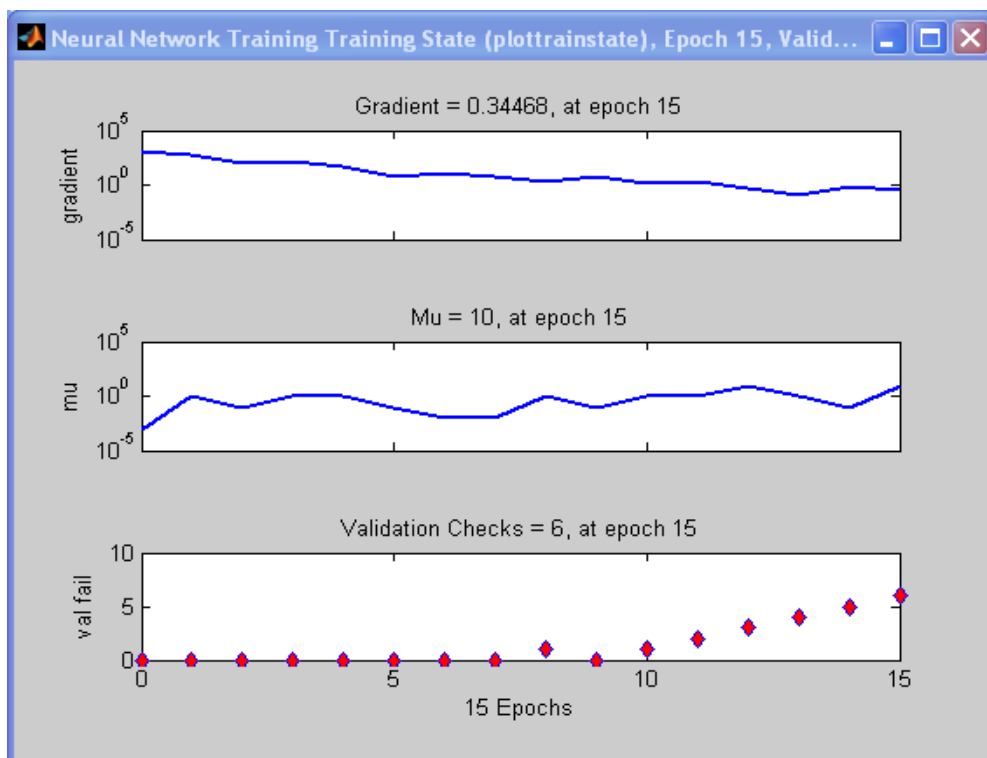
W pierwszym przypadku wykorzystany został opisany algorytm Levenberga-Marquardta stosując w warstwie ukrytej liczbę neuronów równą $N=3$. Schematy przedstawiają otrzymane wyniki:



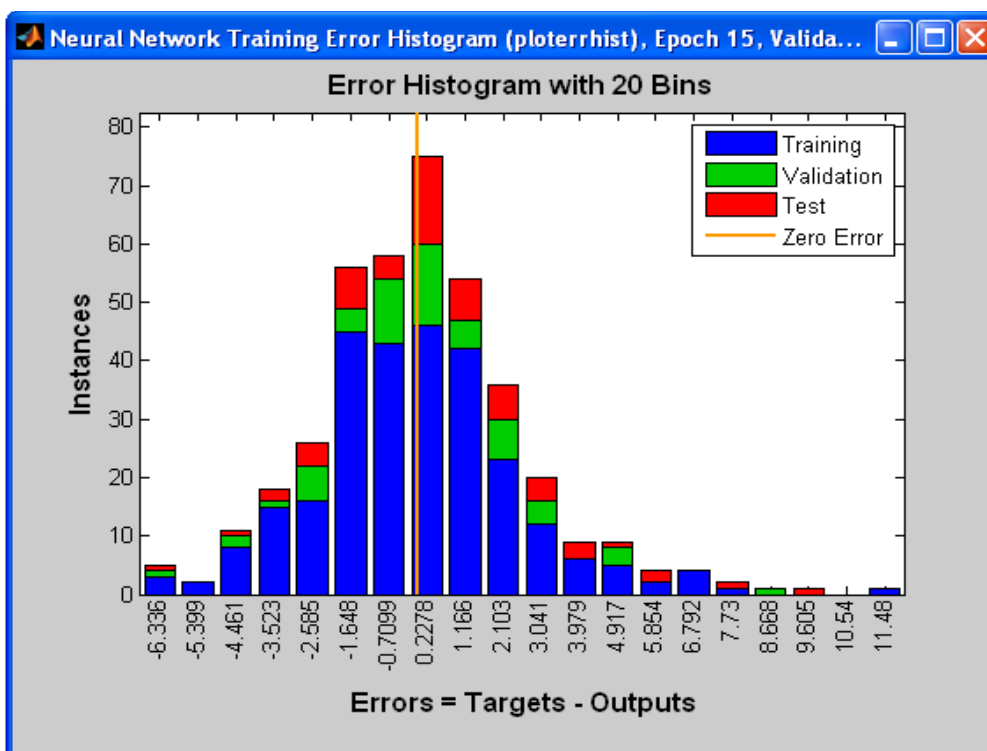
Rys 1. Sieć z zadanymi parametrami. (3 neurony w warstwie ukrytej, zastosowany algorytm LM)



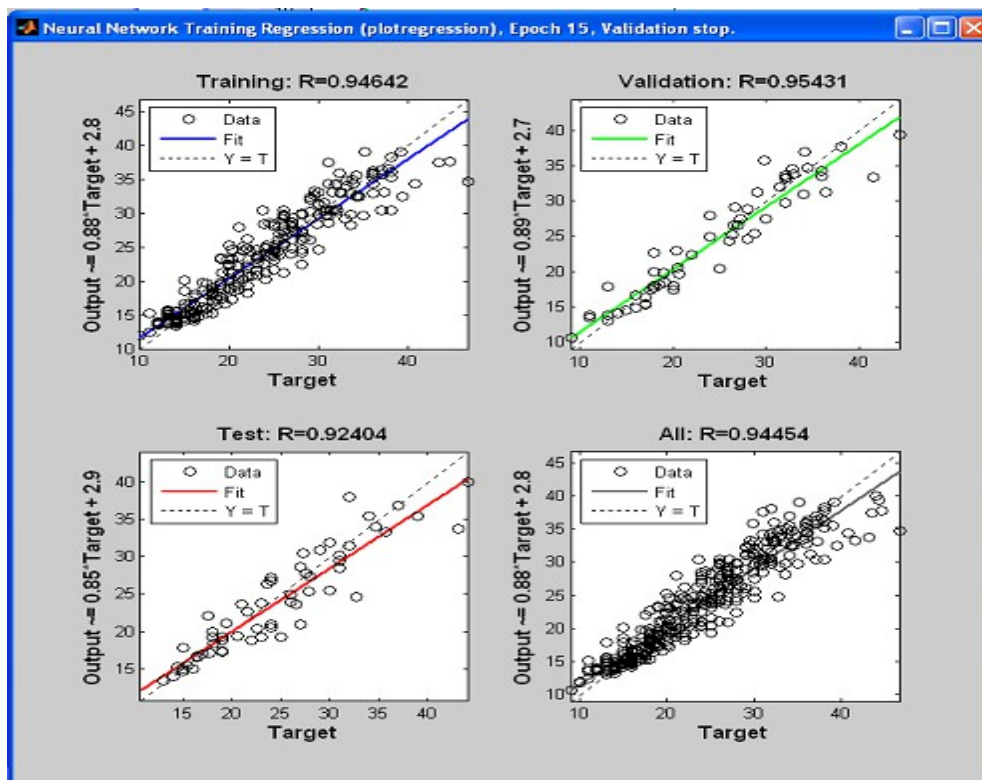
Rys 2. Błąd średnio-kwadratowy w kolejnych epokach przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie LM.



Rys 3. Wartość stanu szkolenia przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie LM.

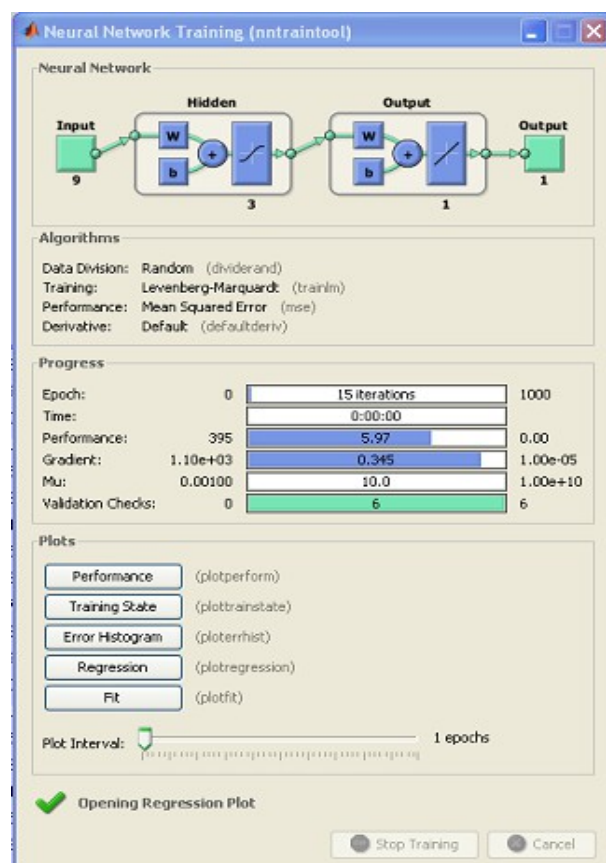


Rys 4. Histogram błędów przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie LM.

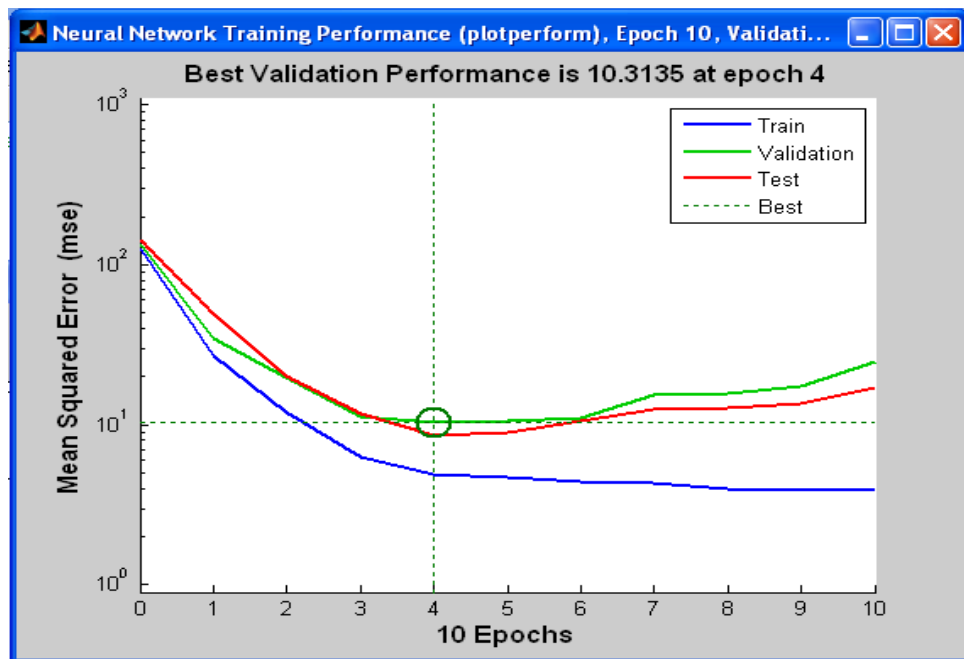


Rys 5. Regresja liniowa przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie LM.

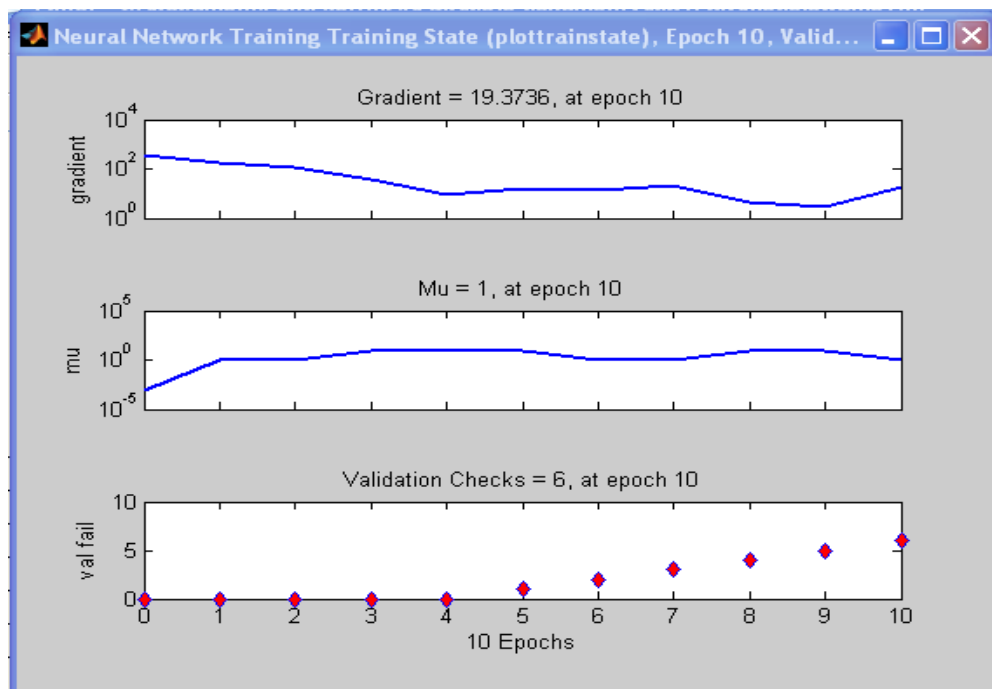
Kolejno w ćwiczeniu została powiększona liczba neuronów do 10-ciu i przy zastosowaniu tej samej metody wygenerowano wyniki:



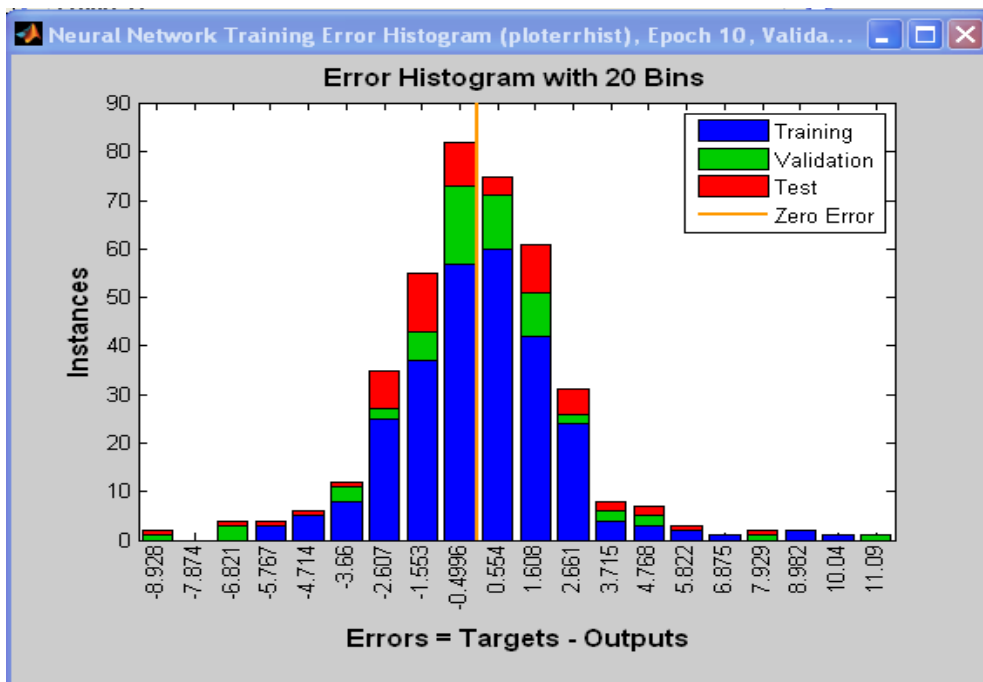
Rys 6. Sieć z zadanymi parametrami.(10 neuronów w warstwie ukrytej, zastosowany algorytm LM)



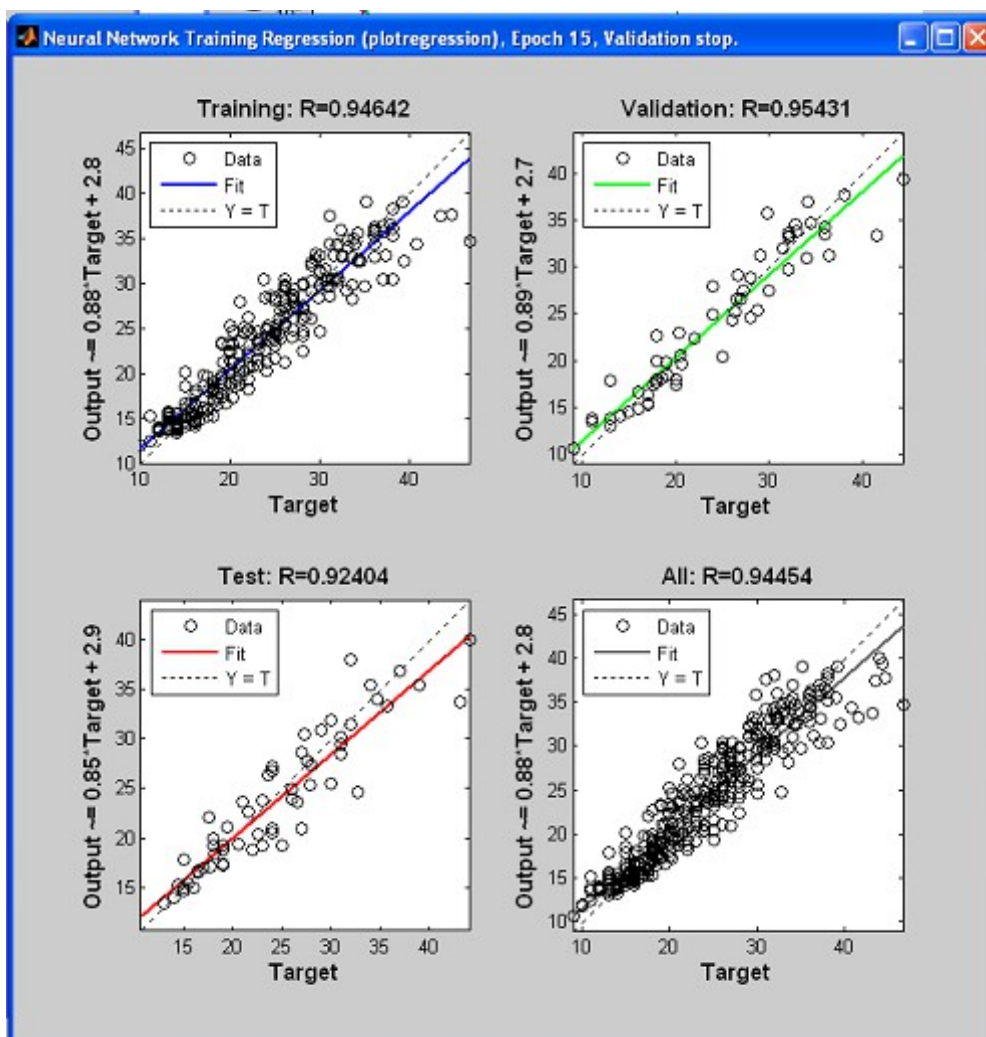
Rys 7. Błąd średnio-kwadratowy w kolejnych epokach przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie LM.



Rys 8. Wartość stanu szkolenia przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie LM.



Rys 9. Histogram błędów przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie LM.



Rys 10. Regresja liniowa przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie LM.

Metoda Momentum BackPropagation

Klasyczny algorytm wstecznej propagacji błędów modyfikuje wagę w_{kj} o wartość (Δw_{kj}) , która jest proporcjonalna do pochodnej cząstkowej funkcji celu (wagi są modyfikowane zgodnie z regułą delty):

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}} = \eta \delta_k y_j$$

Gdzie η jest parametrem nauczania algorytmu wstecznej propagacji błędów.

Jednym z rozwiązań umożliwiających bezpieczne zwiększenie efektywnego tempa uczenia sieci bez pogarszania stabilności procesu jest zastosowanie momentowej metody wstecznej propagacji błędów (ang. Momentum BackPropagation).

Istotą metody jest wprowadzenie do procesu uaktualniania wagi pewnej bezwładności tzw. "momentu", proporcjonalnego do zmiany tej wagi w poprzedniej iteracji:

$$w_{ji}(t+1) = w_{ji}(t) + \eta \delta_j(t) y_i(t) + \underbrace{\mu (w_{ji}(t) - w_{ji}(t-1))}_{\text{Momentum}}$$

gdzie $\mu \in (-0,1]$ - współczynnik momentu.

Pierwsze dwa składniki po prawej stronie wyrażenia są analogiczne, jak w klasycznej metodzie BP, natomiast ostatni uwzględnia poprzednią zmianę wagi i jest niezależny od aktualnej wartości gradientu.

Zasada działania metody:

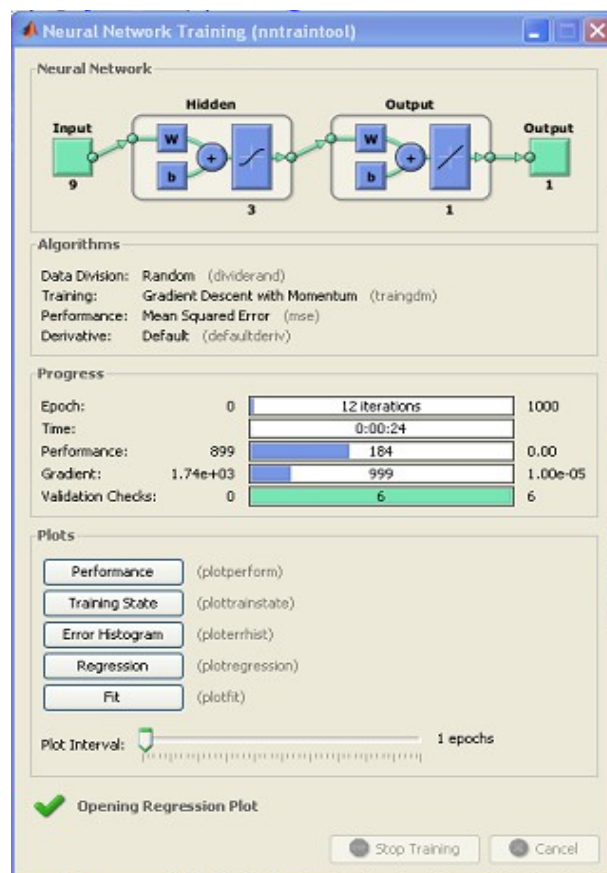
- jeżeli w kolejnych krokach gradienty wykazują ten sam kierunek (pochodna cząstkowa dla danej wagi ma ten sam znak) ich działanie się kumuluje i przyrosty wag są coraz większe.
- jeżeli znaki z kolejnych kroków są różne (gradienty wykazują przeciwny kierunek) działanie jest osłabiane co przeciwdziała powstawaniu oscylacji.

Składnik momentu wpływa pozytywnie w sytuacjach gdy:

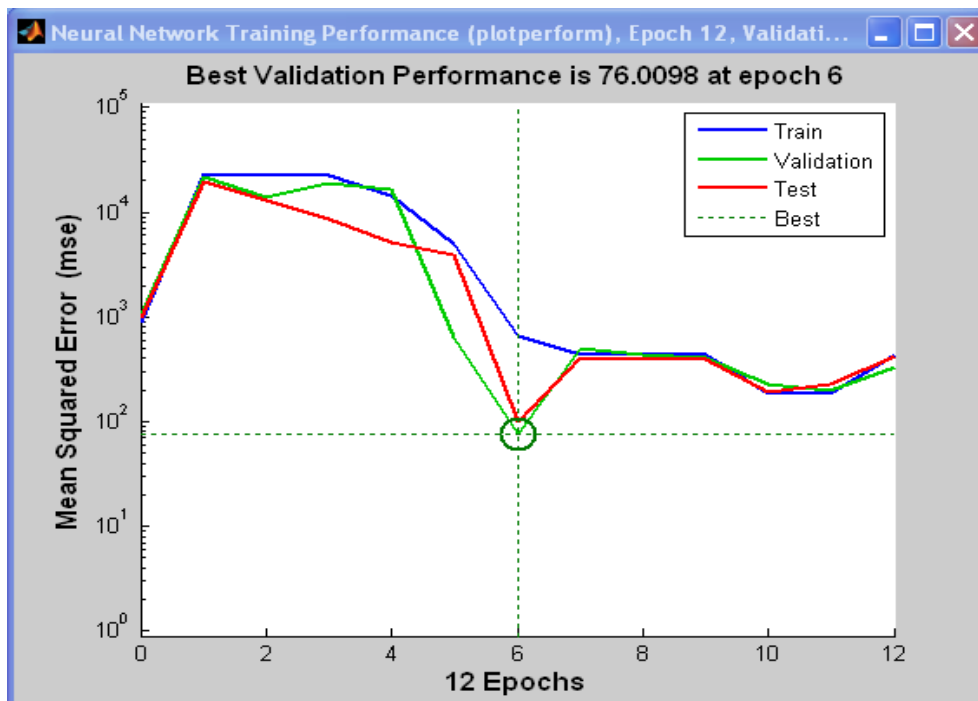
- występują duże płaskie odcinki funkcji błędów – powoduje zwiększenie „kroku”,
- kolejne gradienty są przeciwnie skierowane – powoduje skrócenie „kroku”,
- uczenie „przebiega” przez minimum lokalne funkcji błędów – pomaga go opuścić,
- występują na powierzchni funkcji błędów tzw. wąwozy (wąskie obszary o stromych ścianach bocznych i głębokich dnach).

Składnik momentu nie powinien zdominować procesu uczenia, ponieważ grozi to jego niestabilnością - aby zapobiec temu kontroluje się wartość funkcji błędów w trakcie uczenia, doprowadzając do jej wzrostu jedynie w ograniczonym zakresie.

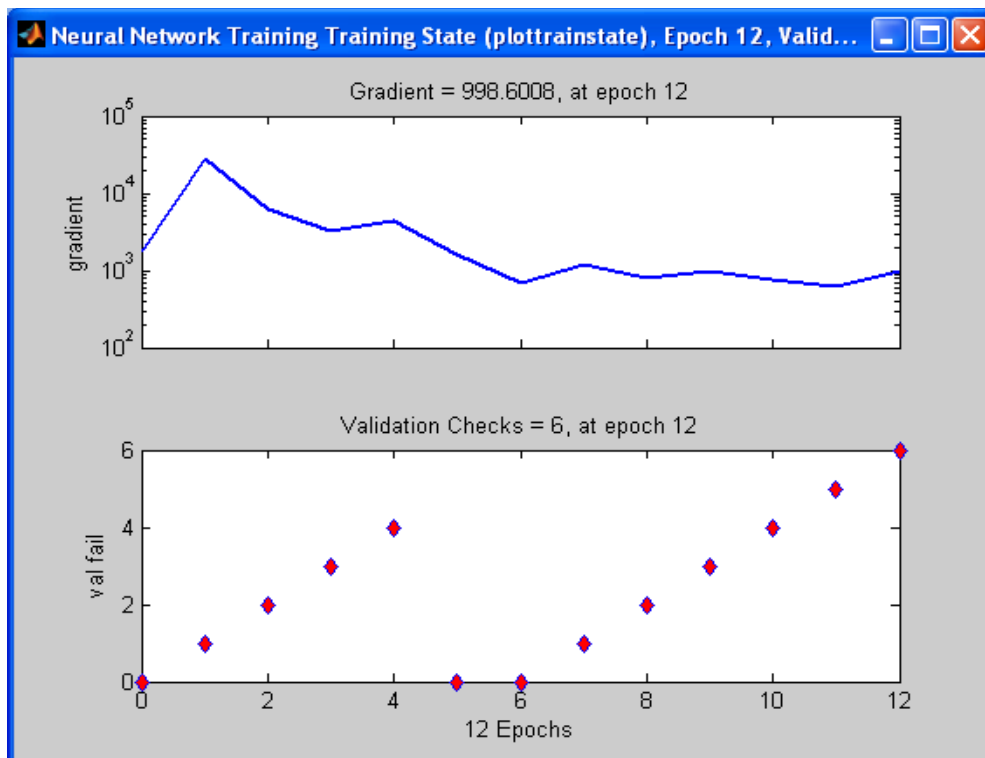
Schematy obrazują otrzymane wyniki uczenia sieci neuronowej zawierającej 3 neurony w warstwie ukrytej i użycia metody Momentum BackPropagation.:



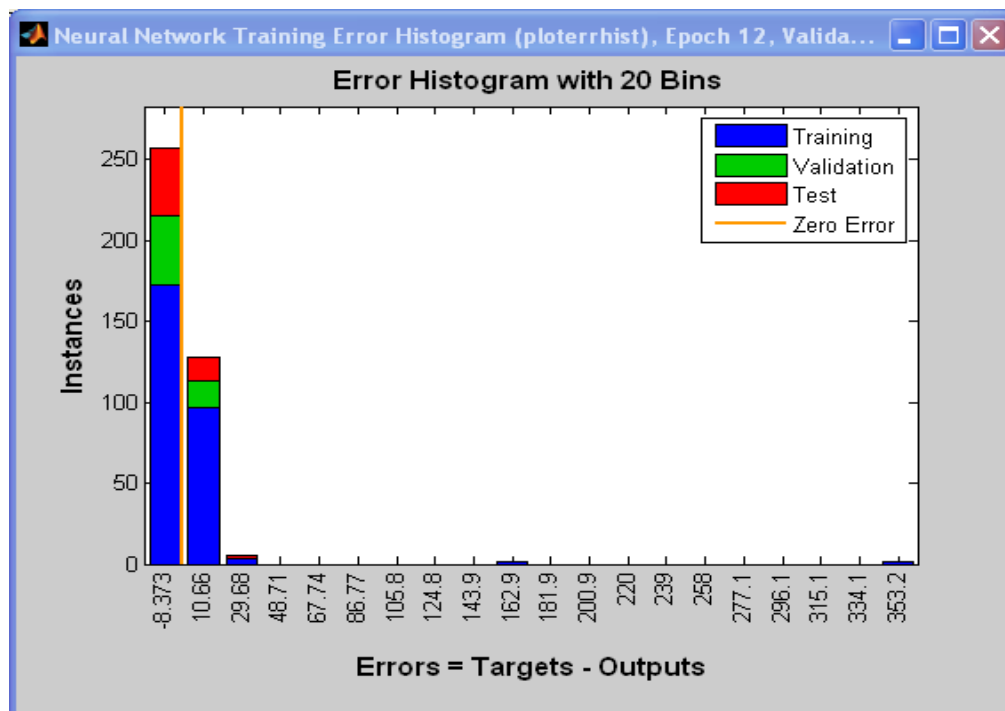
Rys 11. Sieć z zadanymi parametrami. (3 neurony w warstwie ukrytej, zastosowany algorytm Momentum BackPropagation)



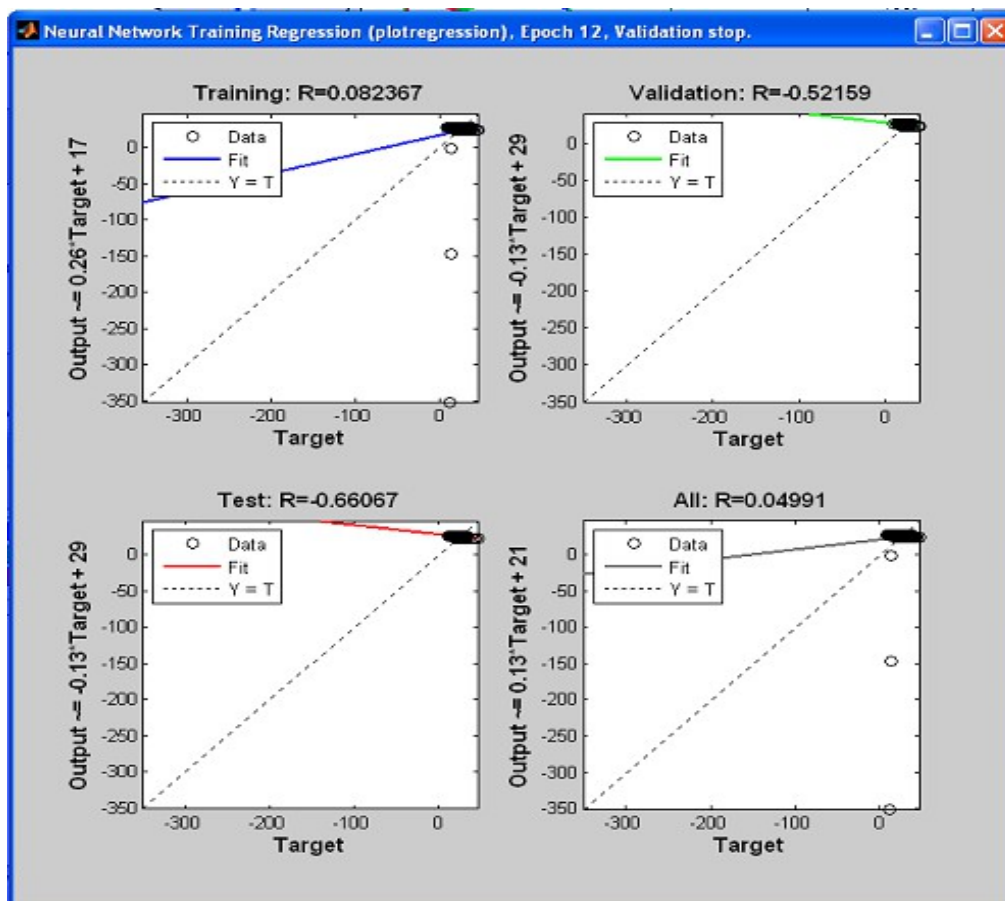
Rys 12. Błąd średnio-kwadratowy w kolejnych epokach przy zadanym 3 neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.



Rys 13. Wartość stanu szkolenia przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.

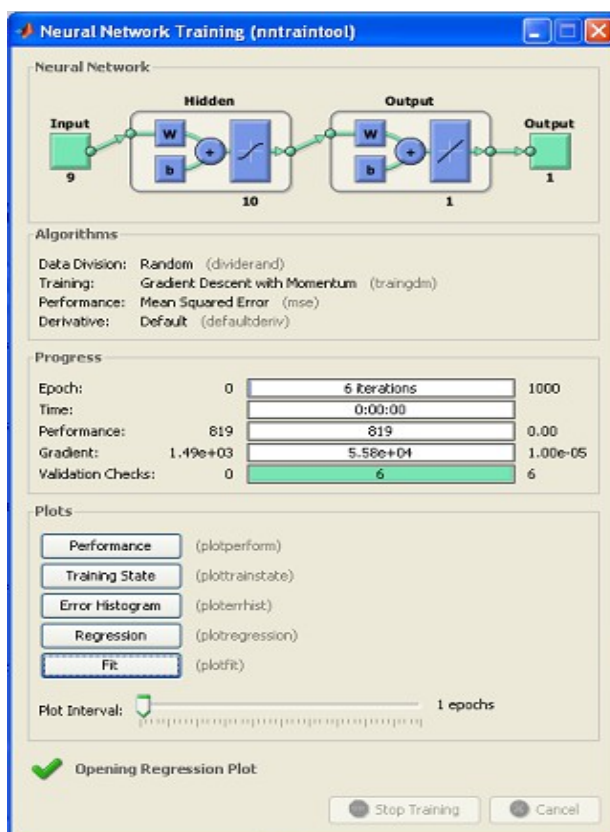


Rys 14. Histogram błędu przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.

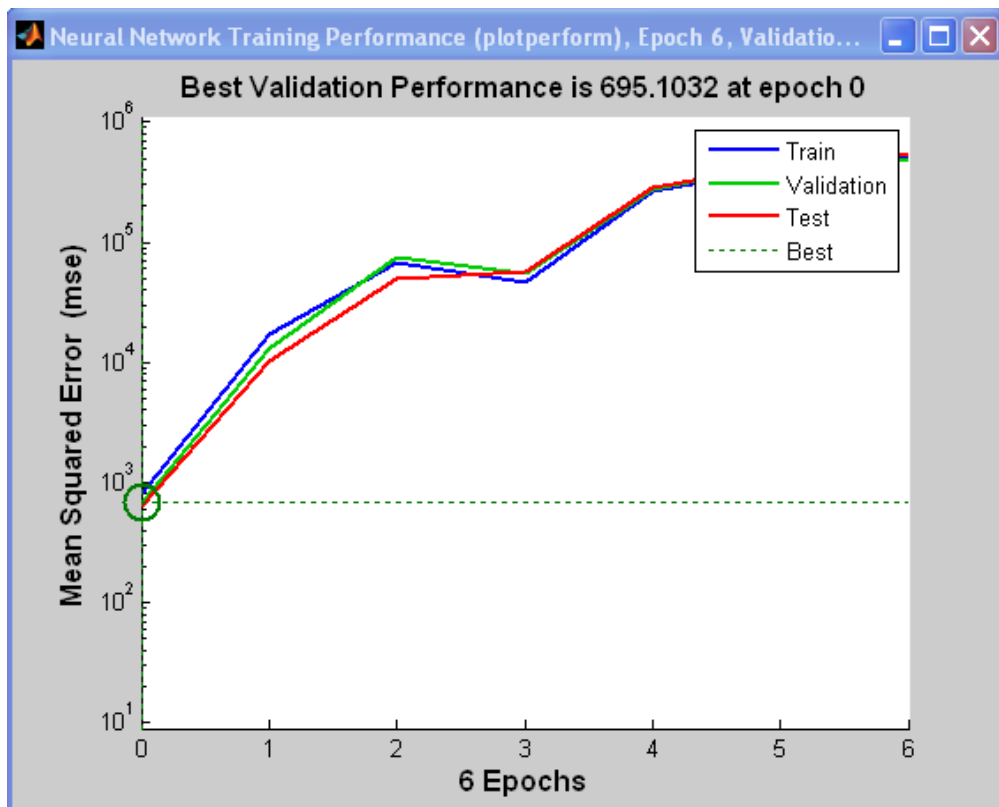


Rys 15. Regresja liniowa przy zadanych 3 neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.

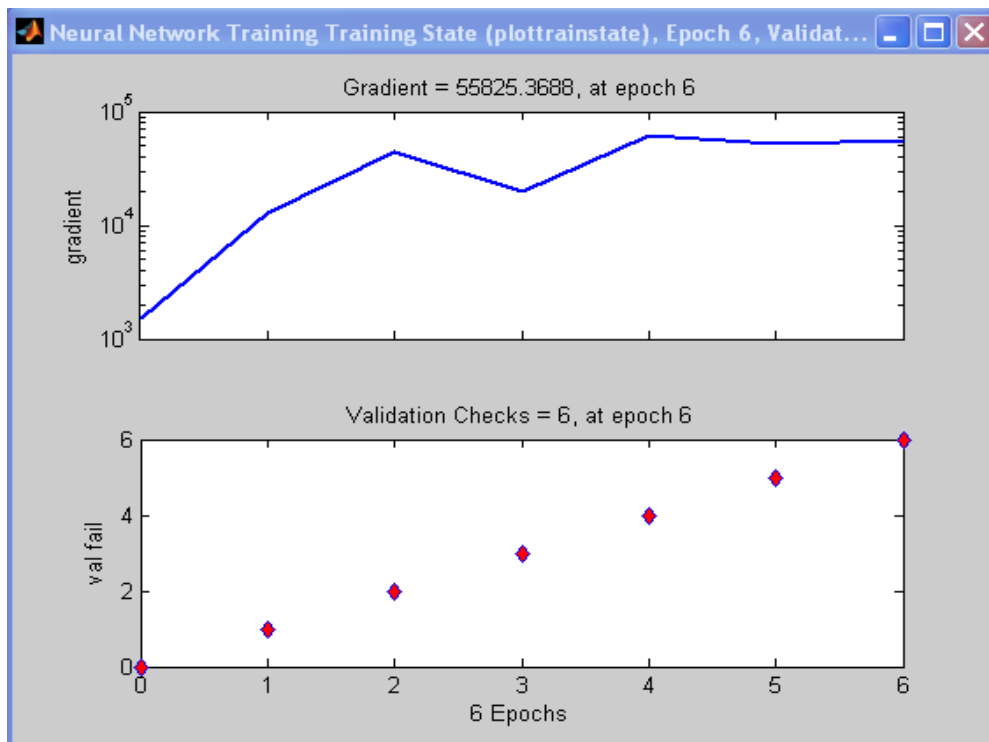
Kolejno w ćwiczeniu została powiększona liczba neuronów do 10-ciu i przy zastosowaniu tej samej metody wygenerowano wyniki:



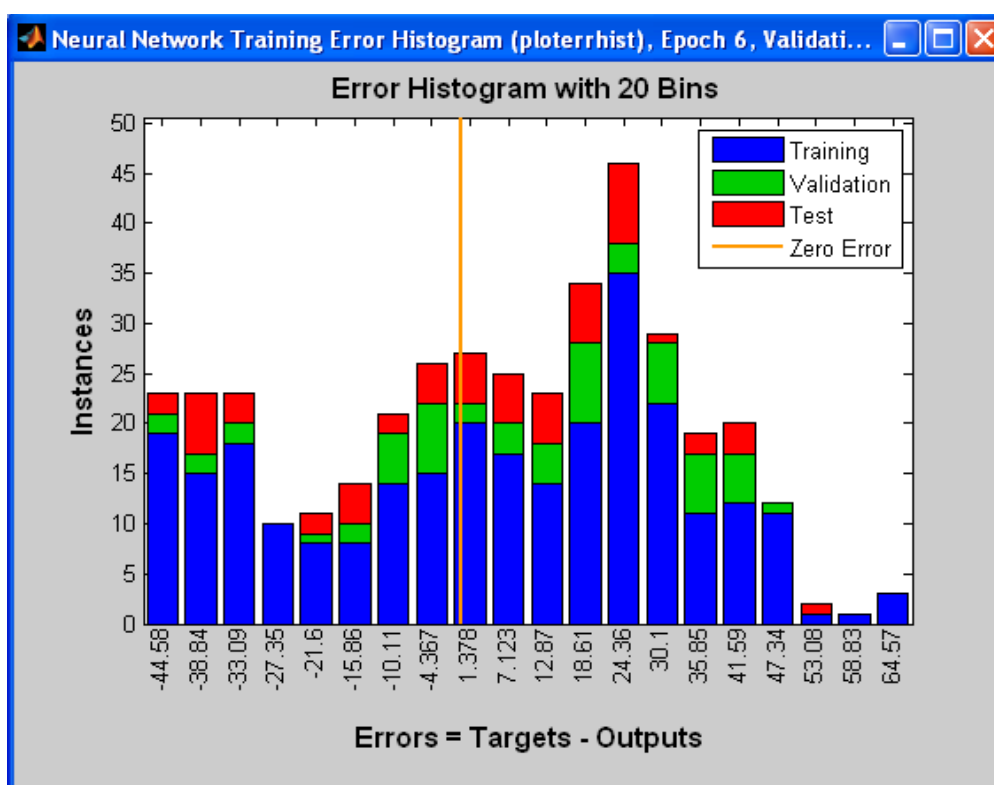
Rys 16. Sieć z zadanymi parametrami. (10 neuronów w warstwie ukrytej, zastosowany algorytm Momentum BackPropagation)



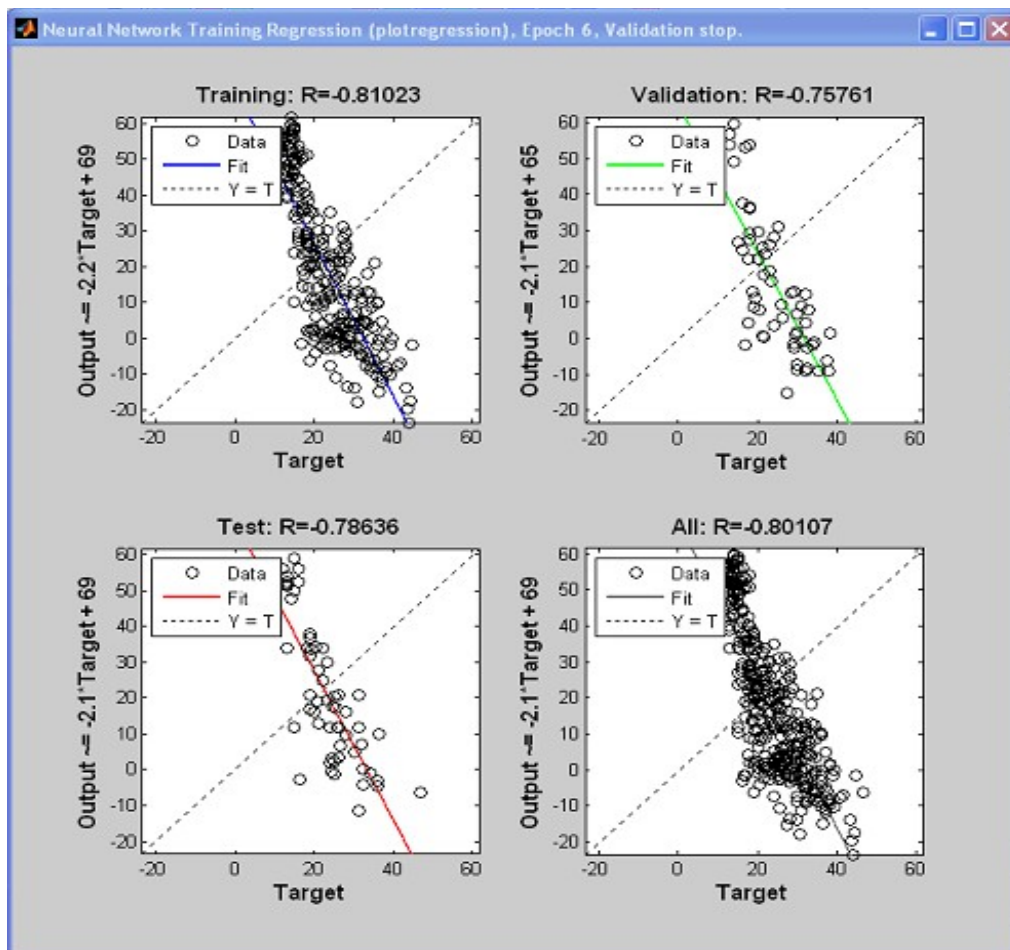
Rys 17. Błąd średnio-kwadratowy w kolejnych epokach przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.



Rys 18. Wartość stanu szkolenia przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.



Rys 19. Histogram błędu przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.



Rys 20. Regresja liniowa przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanym algorytmie Momentum BackPropagation.

Metoda gradientowa Broydena-Fletcher-Goldfarb-Shanno (quasi-Newtonowska)

Jest to algorytm numeryczny mający na celu znalezienie minimum zadanej funkcji celu. Jest to jedna z prostszych metod optymalizacji.

Metody gradientowe polegają na wyznaczaniu kolejnego kierunku poszukiwań na podstawie znajomości gradientu funkcji celu, w punkcie osiągniętym w poprzednim kroku. Funkcja celu musi być więc znana w postaci analitycznej i ograniczoną od dołu funkcją wypukłą klasy C^2 taką, by można ją było przybliżyć formą kwadratową postaci:

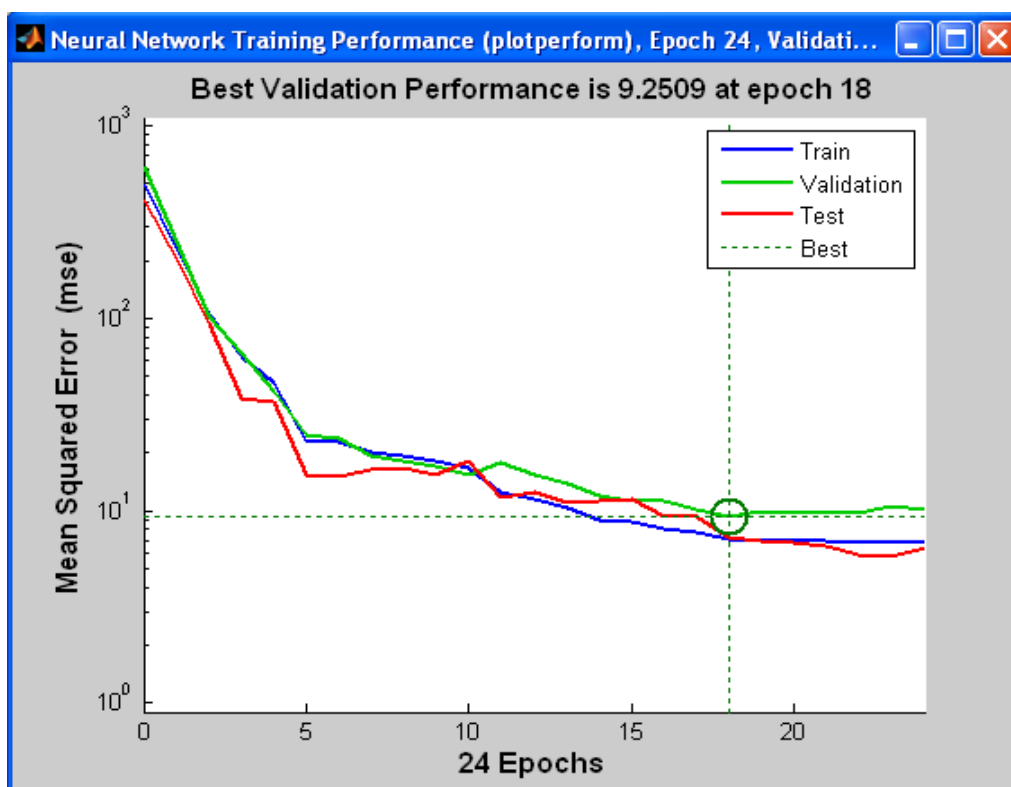
$$f(x) = a + c^T x + \frac{1}{2}(x^T A x)$$

w której macierz A jest symetryczna dodatnio określona, o elementach równych drugim pochodnym cząstkowym funkcji $f(x)$.

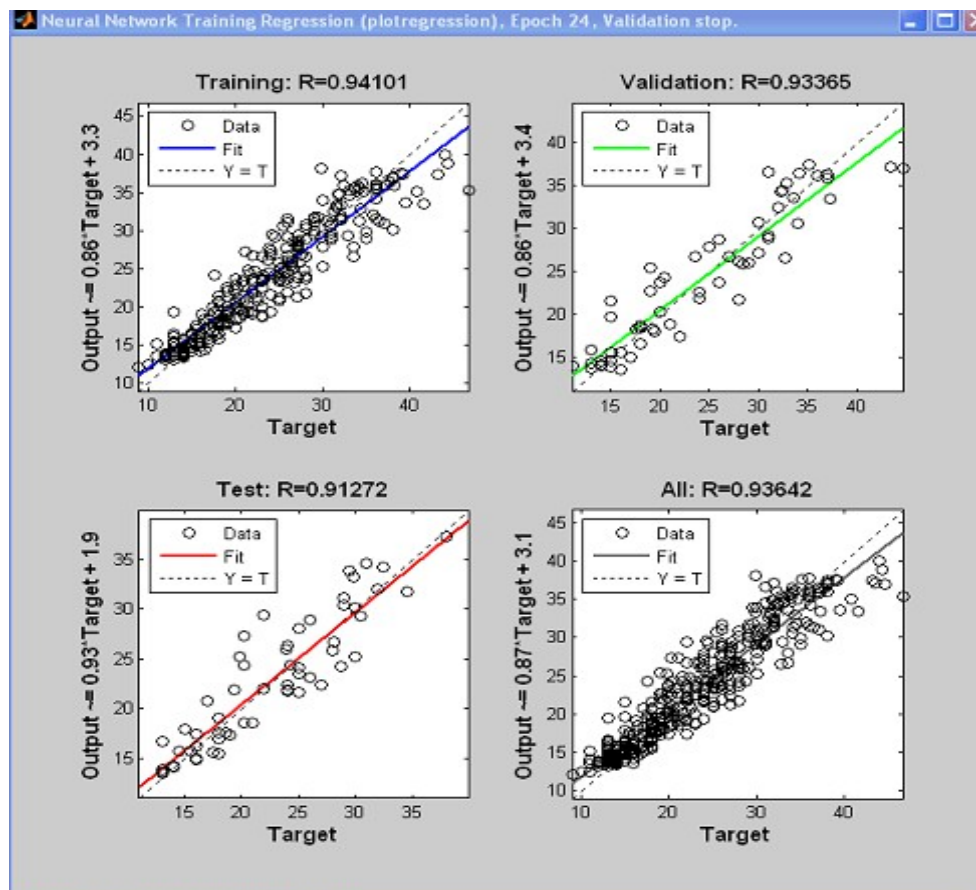
Zasadę działania można przedstawić w kolejnych krokach:

1. Przyjąć punkt startowy x_0 , długość kroku e , współczynnik redukcji kroku α i ϵ , dokładność wyznaczenia ekstremum (zerowania się gradientu). Przyjąć $i=0$.
2. Obliczyć w punkcie x_i wartość funkcji celu $f(x_i)$ i jej gradientu $g(x_i)$.
3. Wyznaczyć kierunek poszukiwań przeciwny do kierunku gradientu $d = -g(x_i)$.
4. Wykonać z punktu x_i krok w wyznaczonym kierunku d o długości e przechodząc do punktu $x_{i+1} = x_i + ed$, czyli do punktu $[x_i - eg(x_i)]$.
5. Obliczyć wartość funkcji celu i jej gradientu w nowym punkcie.
6. Jeśli $g^T g < \epsilon$, zakończyć postępowanie. W przeciwnym razie przejść do punktu 4.
7. Jeśli $f(x_{i+1}) < f(x_i)$, powtórzyć postępowanie dla wyznaczonego punktu x_{i+1} , czyli przyjąć $i=i+1$, przejść do punktu 2.
8. W przypadku przeciwnym cofnąć się do poprzedniego punktu i zmniejszyć krok, czyli przyjąć $e=\alpha e$ i przejść do punktu 4.

Dzięki tak działającej metodzie i ustawieniu 3 neuronów w warstwie ukrytej można było wygenerować następujące schematy przedstawiające otrzymane wyniki:

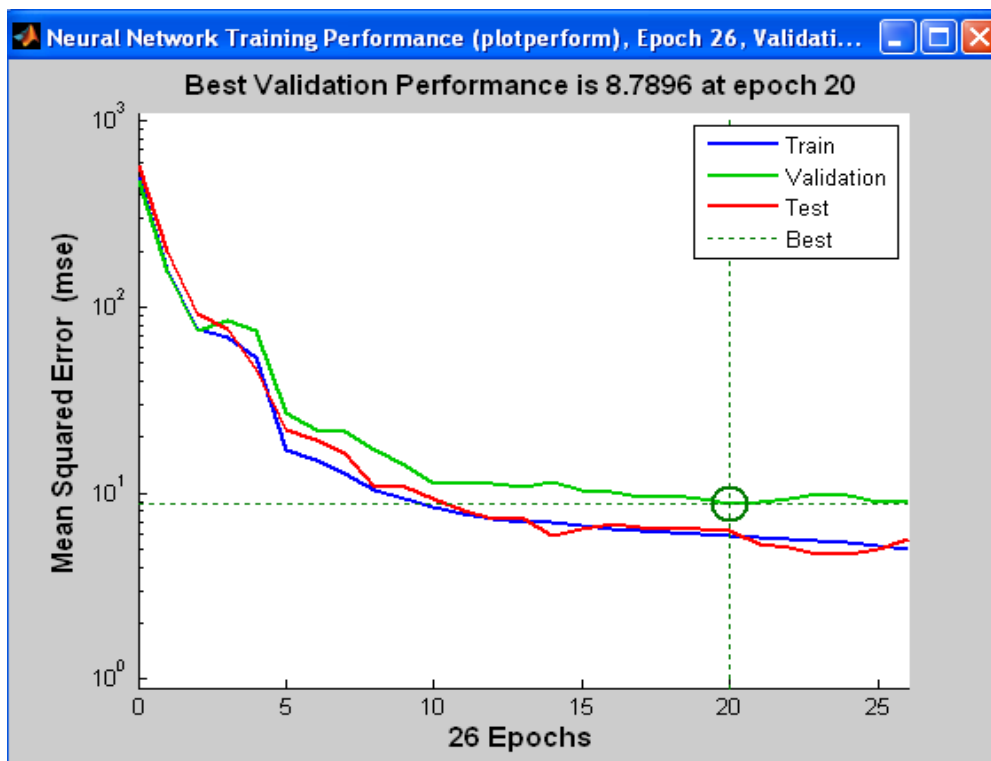


Rys 21. Błąd średnio-kwadratowy w kolejnych epokach przy zadanych 3 neuronach w warstwie ukrytej i zastosowanej metodzie gradientowej.

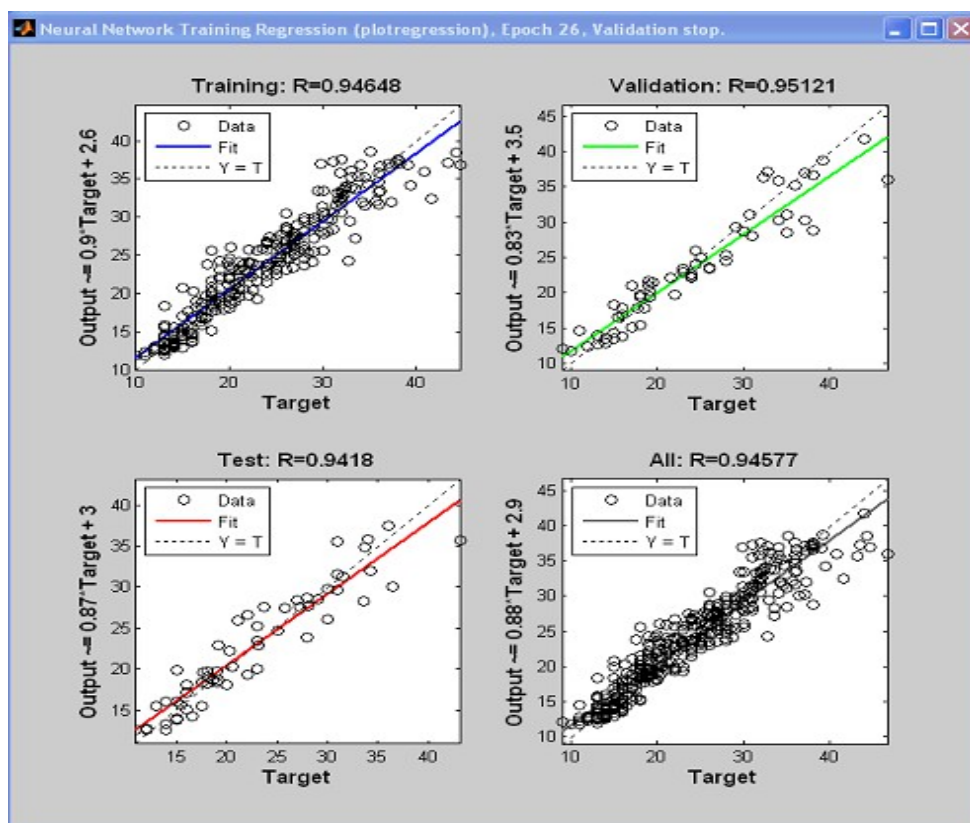


Rys 22. Regresja liniowa w kolejnych epokach przy zadanych 3 neuronach w warstwie ukrytej i zastosowanej metodzie gradientowej.

Kolejno w ćwiczeniu została powiększona liczba neuronów do 10-ciu i przy zastosowaniu tej samej metody wygenerowano wyniki:



Rys 23. Błąd średnio-kwadratowy w kolejnych epokach przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanej metodzie gradientowej.



Rys 24. Regresja liniowa przy zadanych 10-ciu neuronach w warstwie ukrytej i zastosowanej metodzie gradientowej

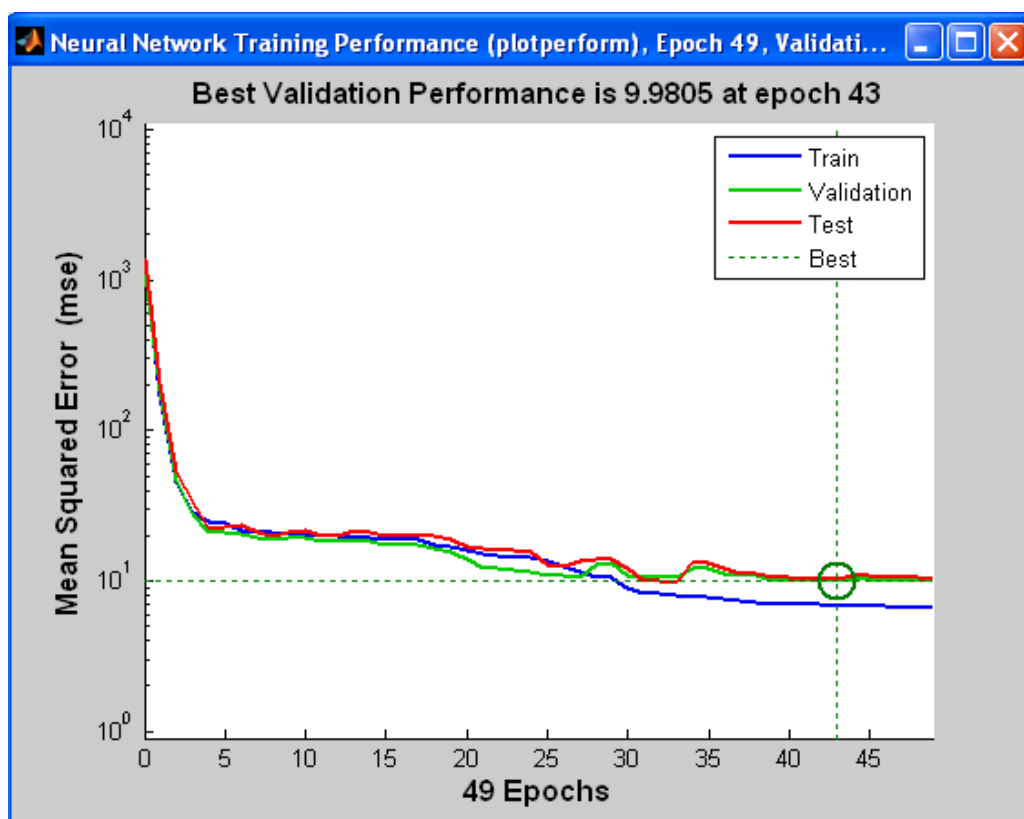
Metoda skalowanego gradientu sprzężonego

W metodzie GS kolejne kierunki poszukiwań są sprzężone do poprzednich względem macierzy A. Z każdego punktu jest wyznaczane minimum funkcji w kierunku.

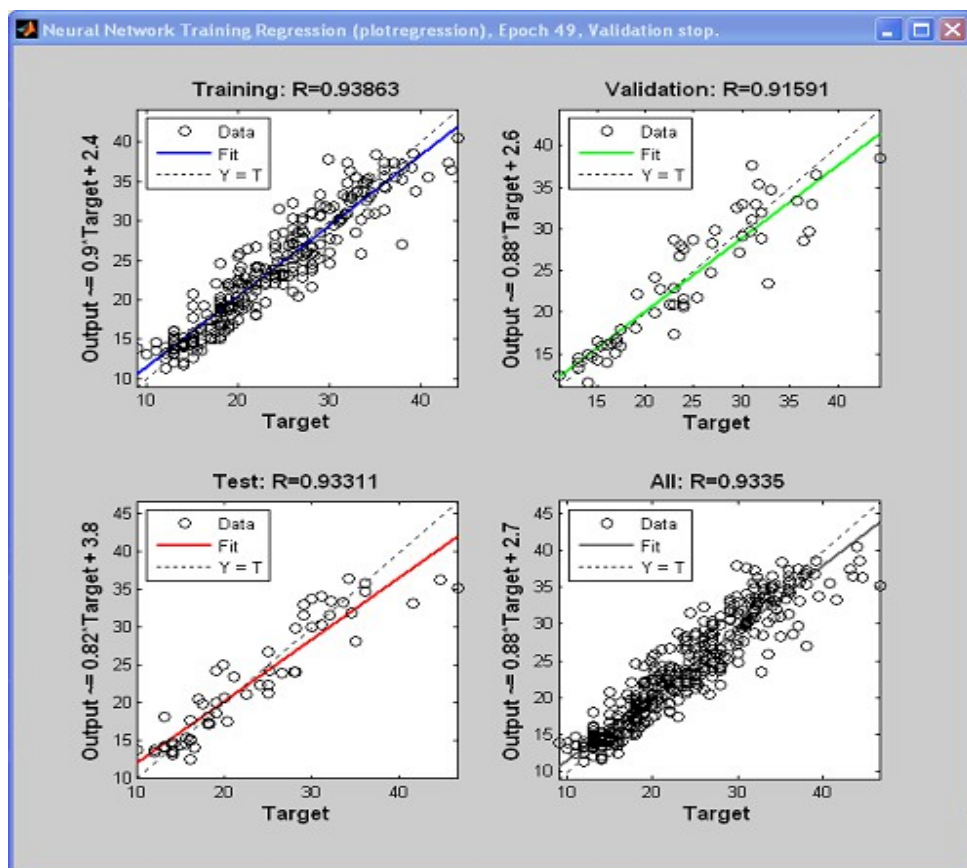
Algorytm metody GS przy poszukiwaniu minimum funkcji $f(x)$ jest następujący:

1. Przyjąć punkt startowy x_0 , dokładność wyznaczenia ekstremum (zerowania się gradientu) ϵ . Przyjąć $i=0$.
2. Obliczyć w punkcie x_i wartość funkcji celu $f(x_i)$ i jej gradientu $g(x_i)$.
3. Wyznaczyć kierunek poszukiwań z_1 przeciwny do kierunku gradientu
 $z_1 = -g(x_1)$, czyli $z_1 = -Ax_0 - c$.
4. W wyniku minimalizacji $f(x)$ w tym kierunku z równania: $z_1^T [A(x_0 + ez_1) + c] = 0$ otrzymuje się wartość kroku e , a następnie punkt $x_1 = x_0 + ez_1$.
5. Obliczyć wartość funkcji celu i jej gradientu w nowym punkcie. Przyjąć $i=i+1$.
6. Wyznaczyć nowy kierunek poszukiwań sprzężony do poprzednich, czyli
 $z_{i+1} = -g(x_i) + \beta z_i$
7. Wykonać z punktu x_i w wyznaczonym kierunku z_{i+1} krok e o takiej wartości, by osiągnąć minimum w tym kierunku, przechodząc do punktu $x_{i+1} = x_i + e z_{i+1}$.
8. Jeśli $g^T g > \epsilon$, przejść do punktu 4. W przeciwnym razie zakończyć postępowanie.

Dzięki tak działającej metodzie i ustawieniu 3 neuronów w warstwie ukrytej można było wygenerować następujące schematy przedstawiające otrzymane wyniki:



Rys 25. Błąd średnio-kwadratowy w kolejnych epokach przy zadanych 3 neuronach w warstwie ukrytej i zastosowanej metodzie gradientu sprzężonego.



Rys 26. Regresja liniowa przy zadanych 3 neuronach w warstwie ukrytej i zastosowanej metodzie gradientu sprzężonego.

3) Wnioski